Memory Access

CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

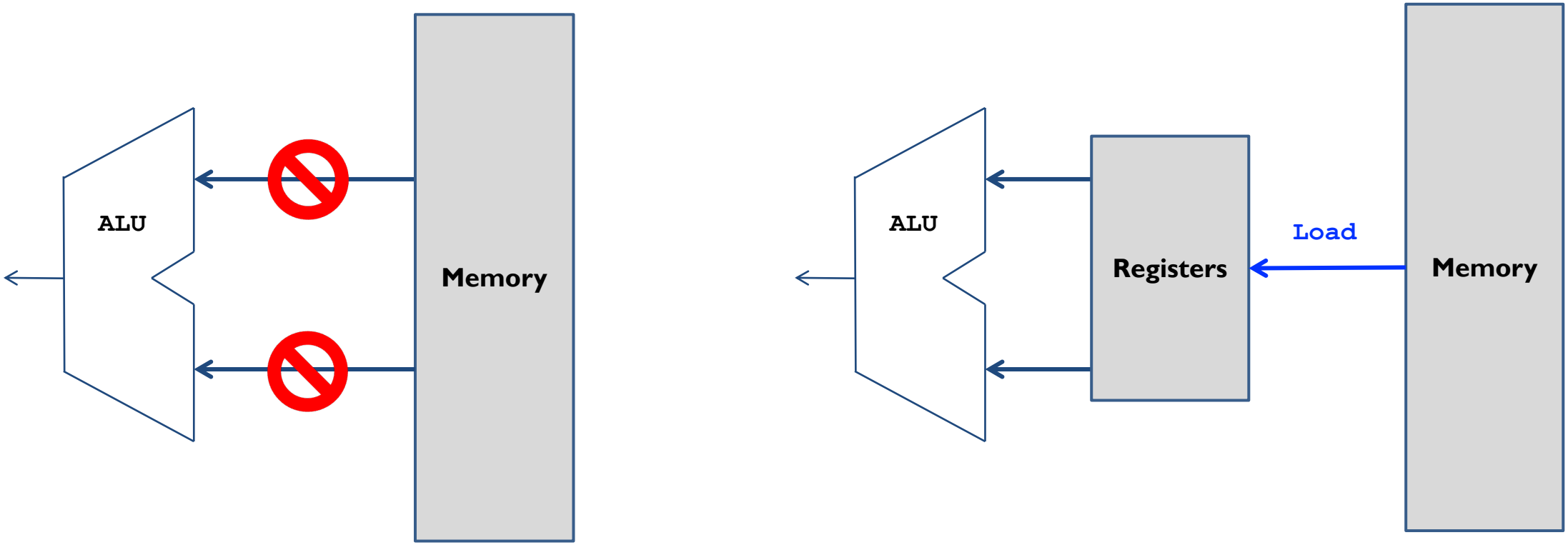
Taken from Dr. Yifeng Zhu's Lecture Notes

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 5

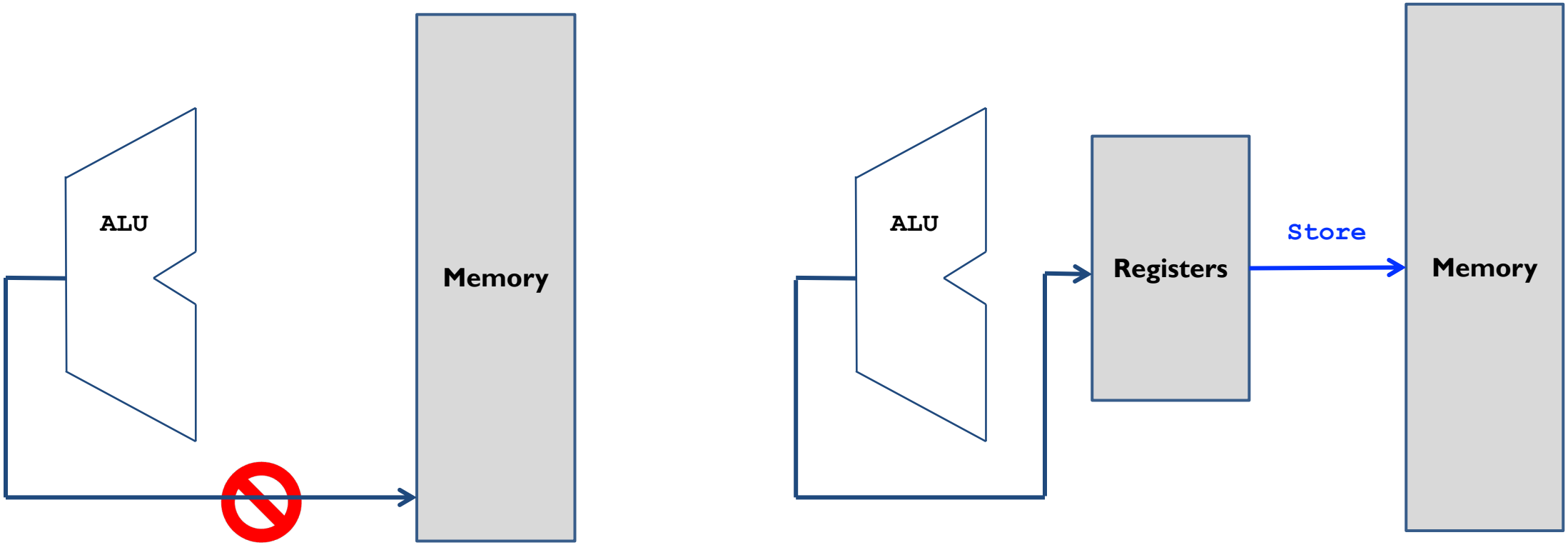
Overview

- How is data addressed?
  - Register offset
    - `LDR r1, [r0, r3]` `; offset = r3`
    - `LDR r1, [r0, r3, LSL #2]` `; offset = r3 * 4`
  - Immediate offset
    - Pre-index: `LDR r1, [r0, #4]`
    - Post-index: `LDR r1, [r0], #4`
    - Pre-index with update: `LDR r1, [r0, #4]!`

Loading Data from Memory



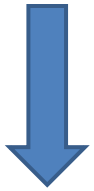
Storing Data to Memory



Load-Modify-Store

C statement

```
x = x + 1;
```

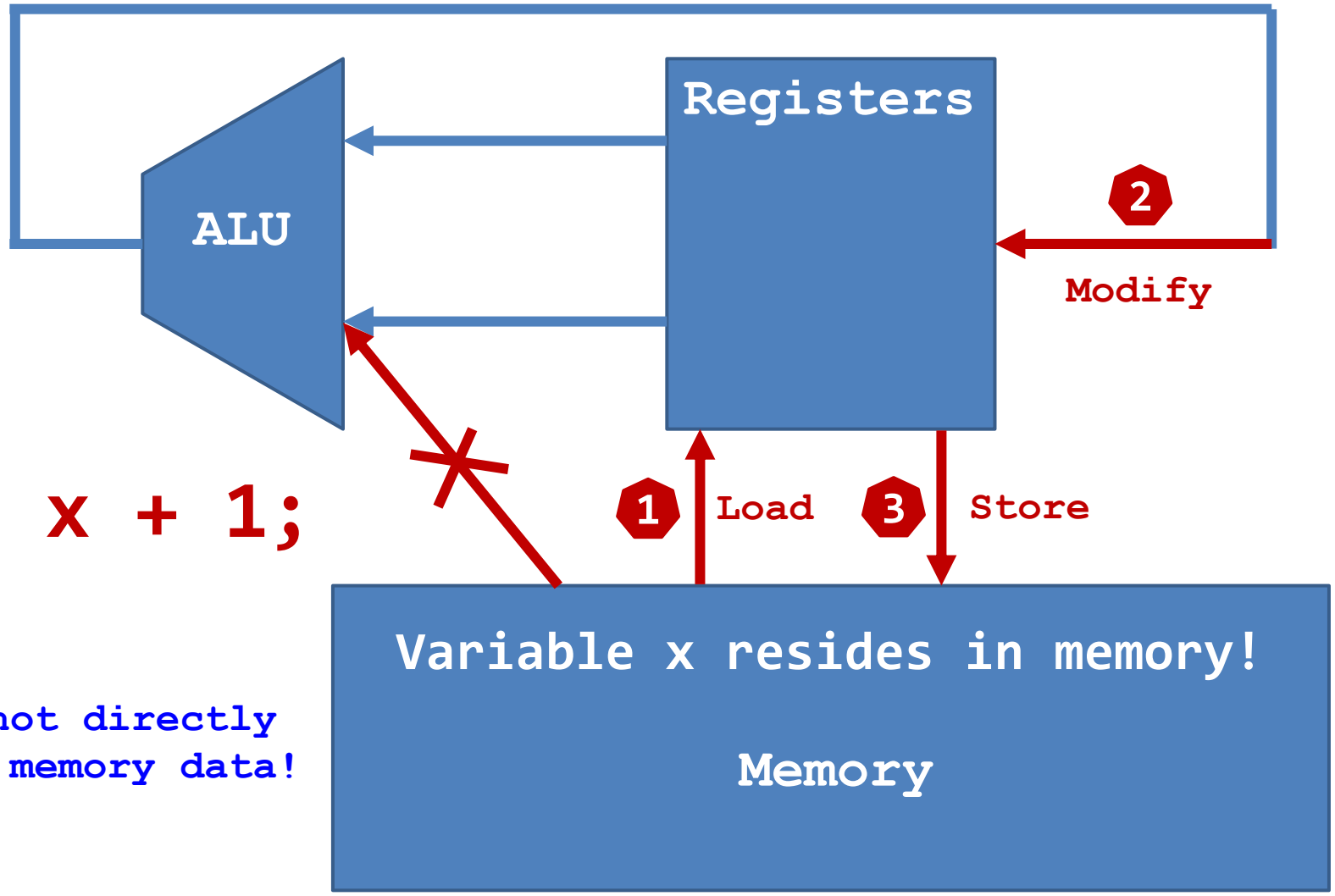


Assume variable X resides in memory and is a 32-bit integer

```
; Assume the memory address of x is stored in r1

LDR r0, [r1]      ; load value of x from memory
ADD r0, r0, #1    ; x = x + 1
STR r0, [r1]      ; store x into memory
```

3 Steps: Load, Modify, Store



Load Instructions

- **LDR rt, [rs]**
  - **Read from memory**
  - Mnemonic: **L**oa**D** to **R**egister (**LDR**)
  - rs specifies the memory address
  - rt holds the 32-bit value fetched from memory

  - For Example:

```
; Assume r0 = 0x08200004
; Load a word:
LDR r1, [r0]          ; r1 = Memory.word[0x08200004]
```

Store Instructions

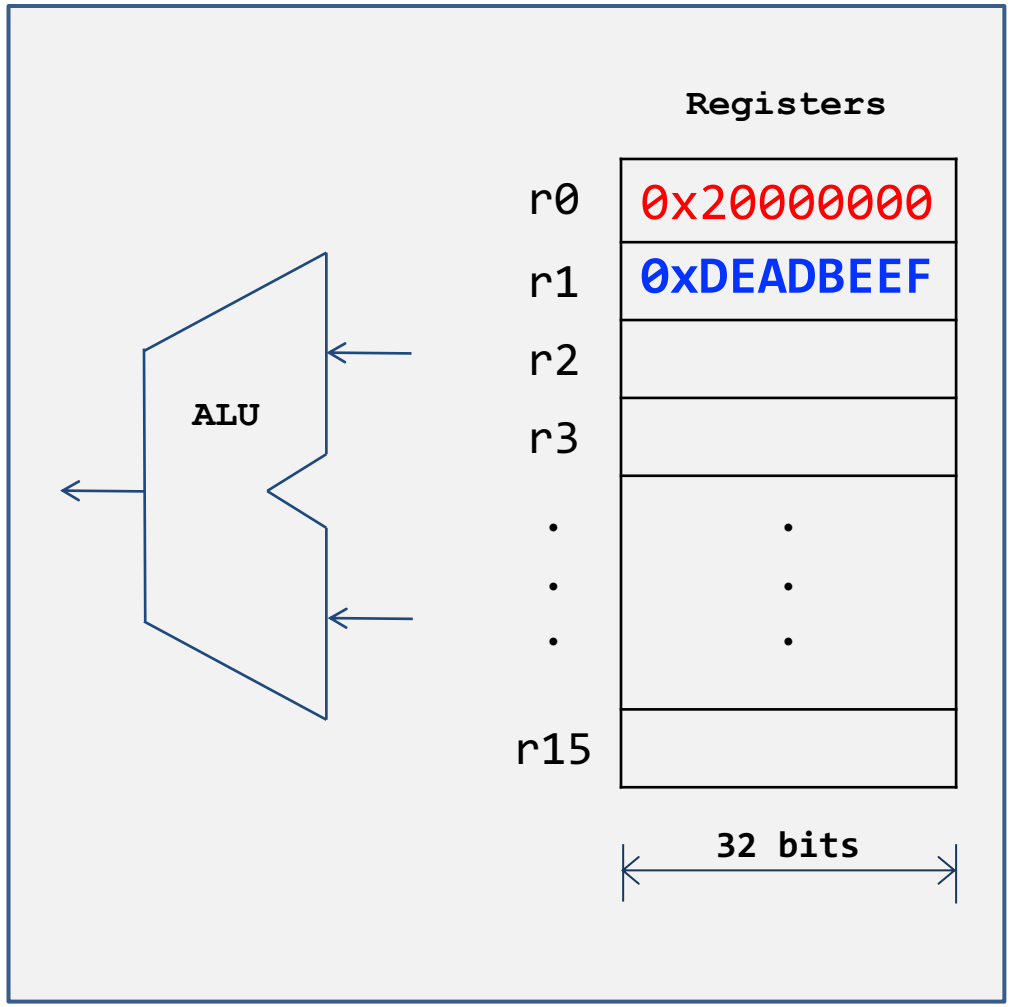
- **STR rt, [rs]**
  - **Write into memory**
  - Mnemonic: **ST**ore from **R**egister (**STR**)
  - rs specifies memory address
  - Save the content of rt into memory

  - For Example:

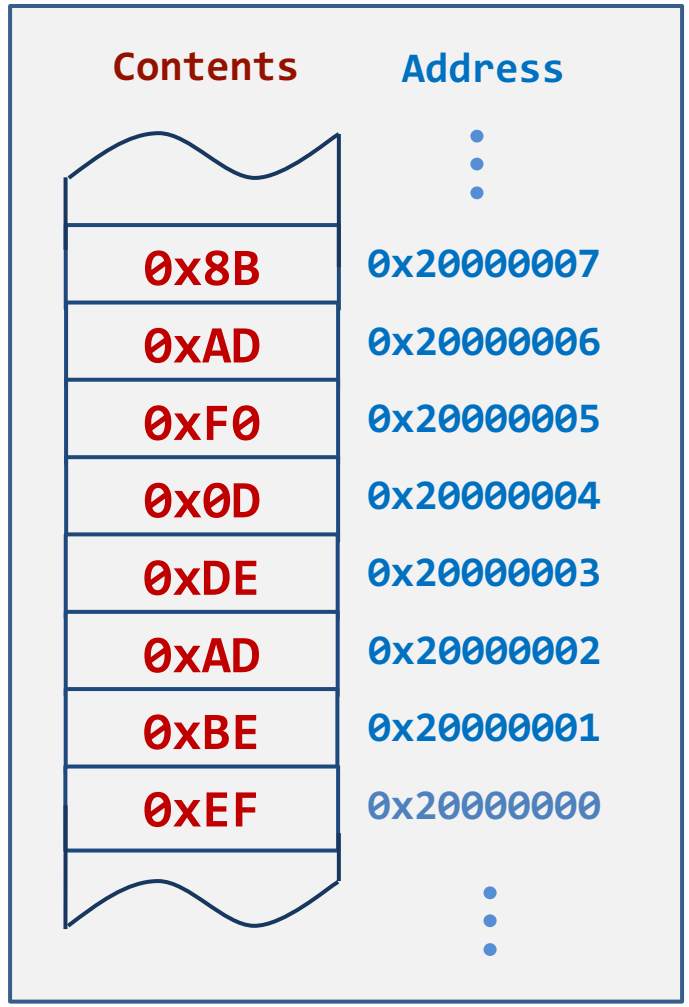
```
; Assume r0 = 0x08200004
; Store a word
STR r1, [r0]     ; Memory.word[0x08200004] = r1
```

Loading Word from Memory

```
LDR r1, [r0] ; r1 = memory.word[r0]
             ; LDR stands for Load to Register
```



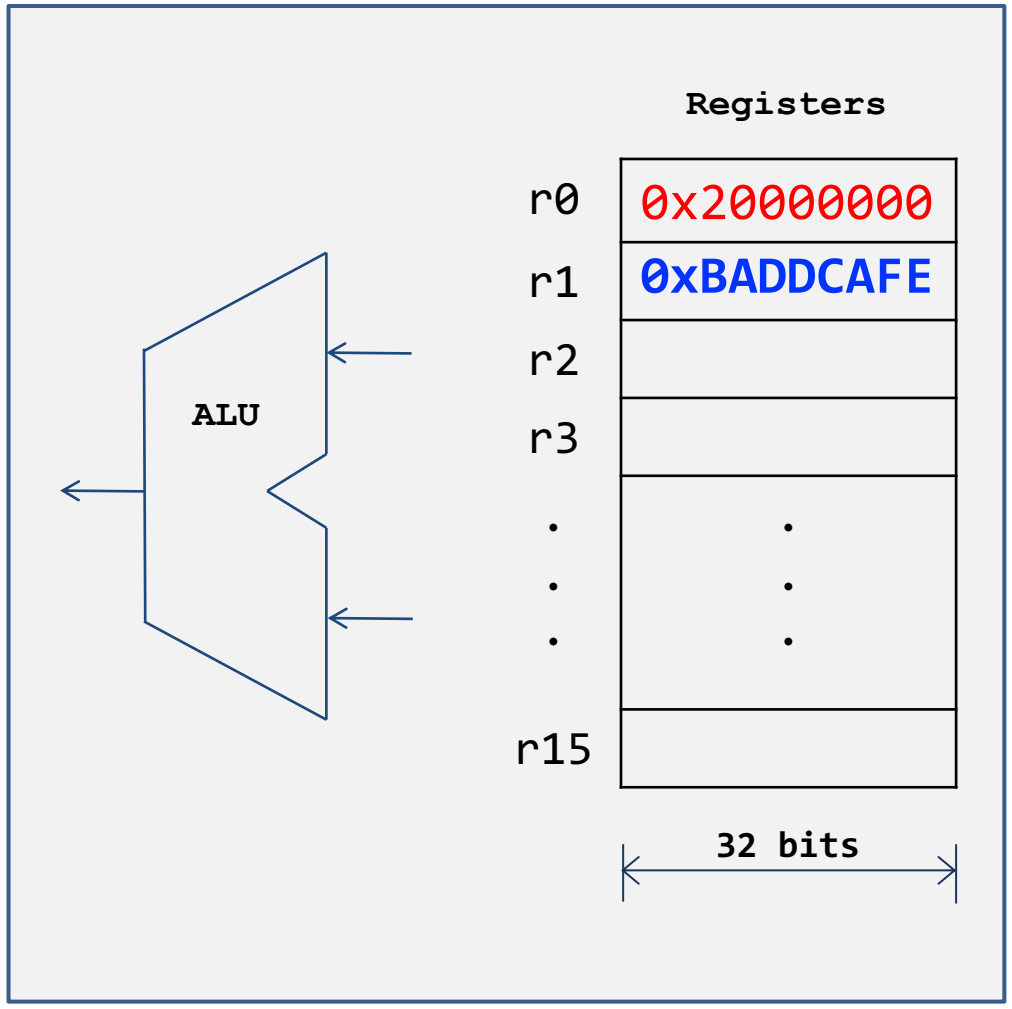
Processor Core



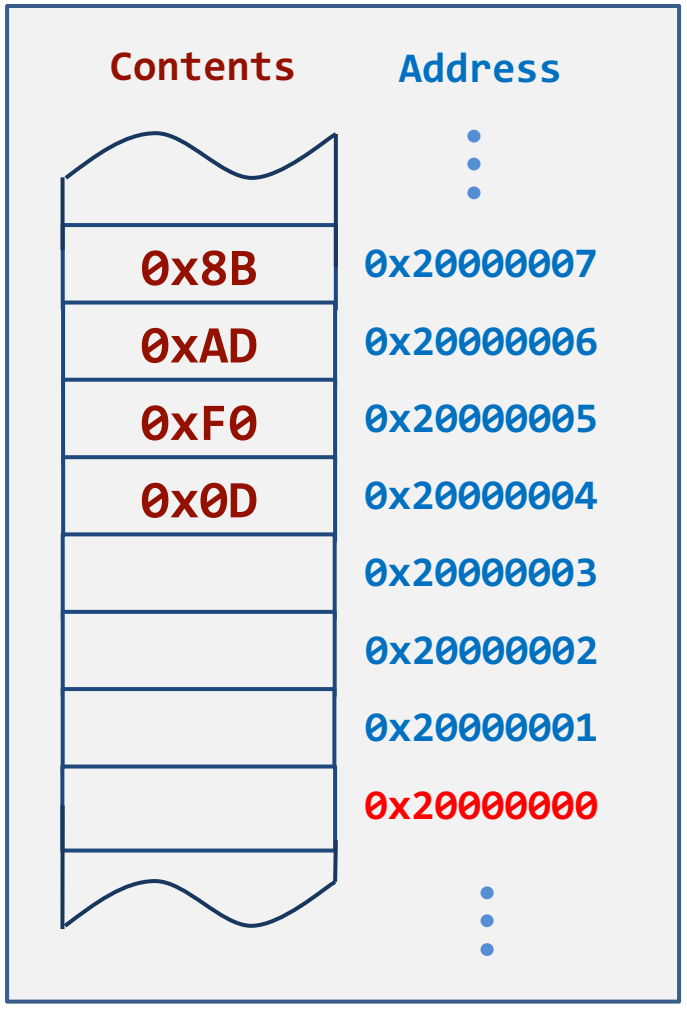
Memory

Storing Word to Memory

```
STR r1, [r0] ; memory.word[r0] = r1
; STR stands for Store Register
```



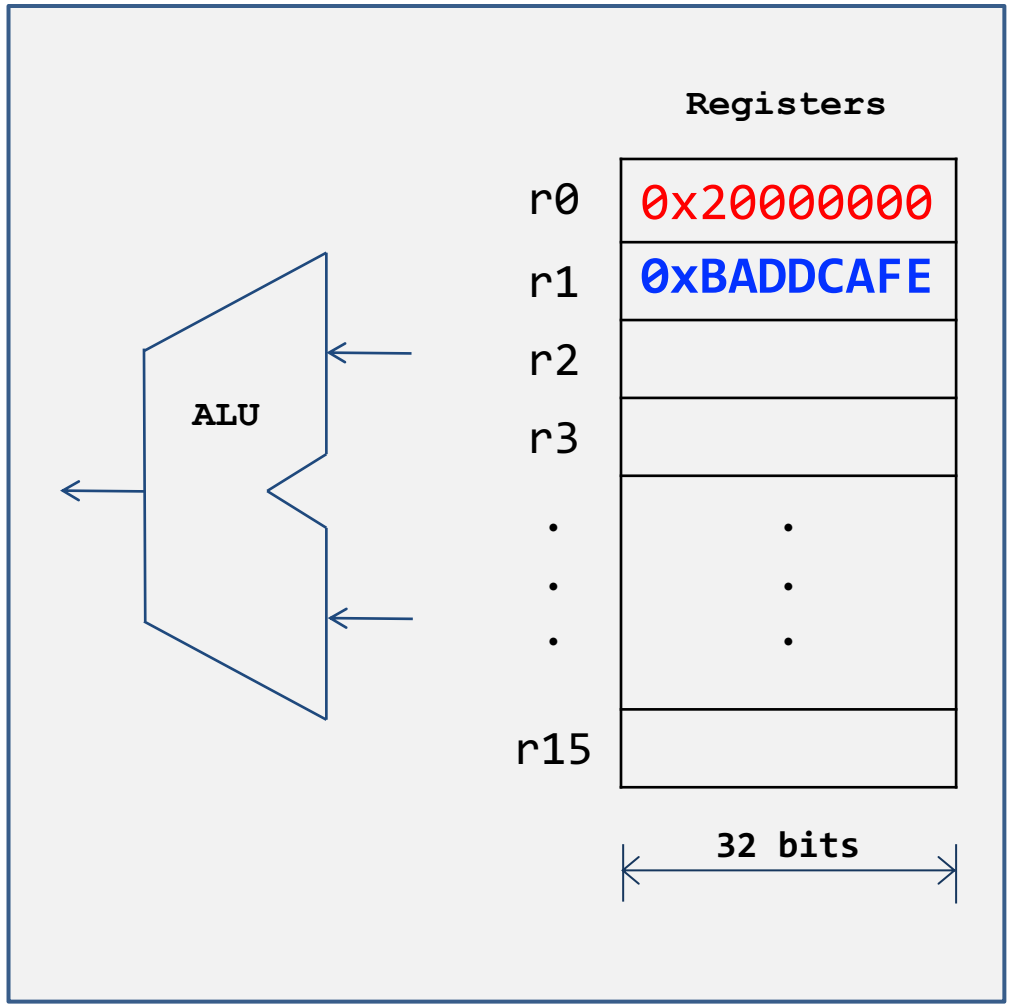
Processor Core



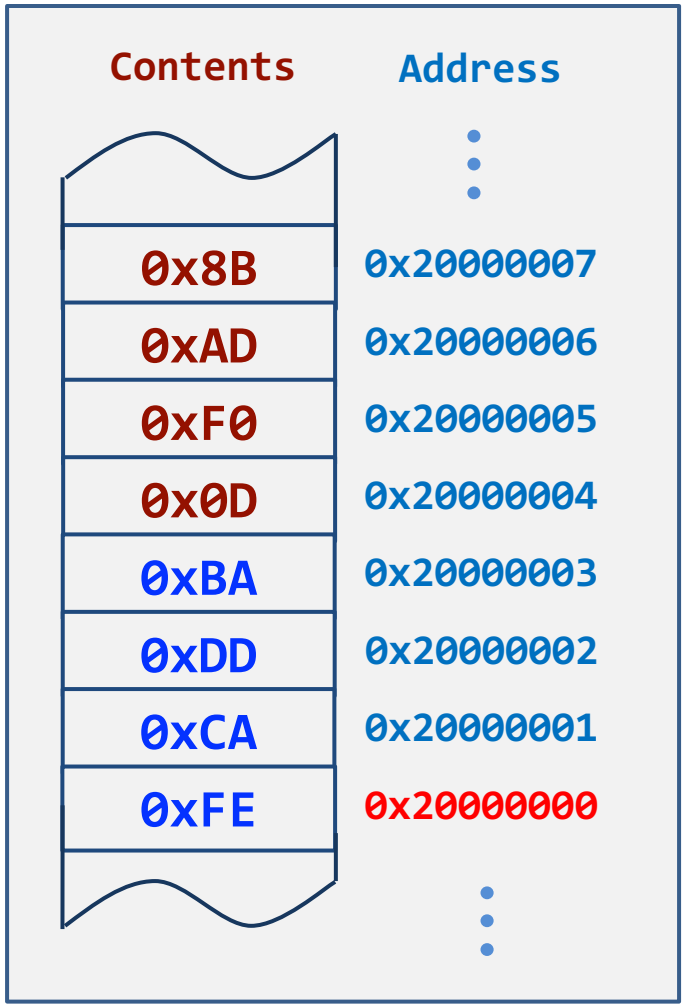
Memory

Storing Word to Memory

```
STR r1, [r0] ; memory.word[r0] = r1
; STR stands for Store Register
```



Processor Core



Memory

Load/Store a Byte, Halfword, Word

```
LDRxxx R0, [R1]
; Load data from memory into a 32-bit register
```

LDR	Load Word	uint32_t/int32_t	unsigned or signed int
LDRB	Load **B**yte	uint8_t	unsigned char
LDRH	Load **H**alfword	uint16_t	unsigned short int
LDRSB	Load **S**igned **B**yte	int8_t	signed char
LDRSH	Load **S**igned **H**alfword	int16_t	signed short int

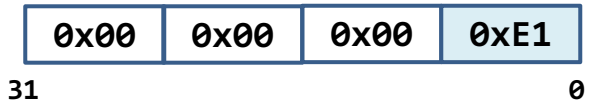
```
STRxxx R0, [R1]
; Store data extracted from a 32-bit register into memory
```

STR	Store Word	uint32_t/int32_t	unsigned or signed int
STRB	Store Lower **B**yte	uint8_t/int8_t	unsigned or signed char
STRH	Store Lower **H**alfword	uint16_t/int16_t	unsigned or signed short

Load a Byte, Half-word, Word

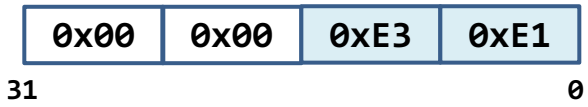
Load a Byte

```
LDRB r1, [r0]
```



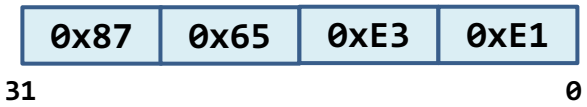
Load a Halfword

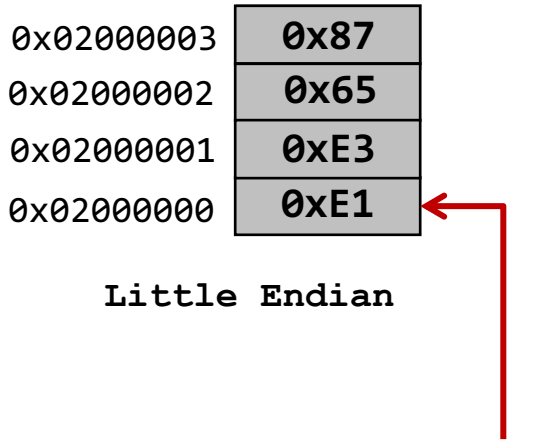
```
LDRH r1, [r0]
```



Load a Word

```
LDR r1, [r0]
```





Sign Extension

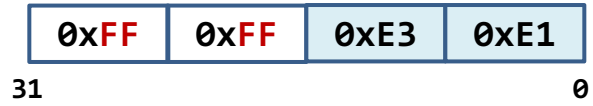
Load a Signed Byte

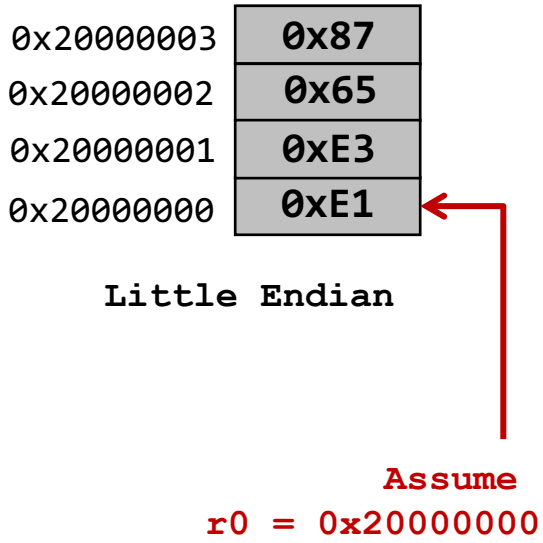
```
LDRSB r1, [r0]
```



Load a Signed Halfword

```
LDRSH r1, [r0]
```





Facilitate subsequent 32-bit signed arithmetic!

Address Modes: Offset in Register

- Address accessed by **LDR/STR** is specified by a base register plus an offset
- Offset can be hold in **a register**

LDR r0,[r1,r2]

- Base memory address hold in register r1
- Offset hold r2
- Target address = r1 + r2

LDR r0,[r1,r2,LSL #2]

- Base memory address hold in register r1
- Offset = r2, LSL #2
- Target address = r1 + r2 * 4

Address Modes: Immediate Offset

- Address accessed by **LDR/STR** is specified by a base register plus an offset
- Offset can be **an immediate value**

```
LDR r0,[r1,#8]
```

- Base memory address hold in register r1
- Offset is an immediate value
- Target address = r1 + 8

Three modes for immediate offset:

- Pre-index,
- Post-index,
- Pre-index with Update

Addressing Mode: Pre-index *vs* Post-index

- Pre-index

```
LDR r1, [r0, #4]
```

- Post-index

```
LDR r1, [r0], #4
```

- Pre-index with Update

```
LDR r1, [r0, #4]!
```

Pre-index

Pre-Index: `LDR r1, [r0, #4]`

Offset: range is -255 to +255

Assume: r0 = 0x20008000

Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

Pre-index

Pre-Index: `LDR r1, [r0, #4]`

Offset: range is -255 to +255

Assume: r0 = 0x20008000

Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

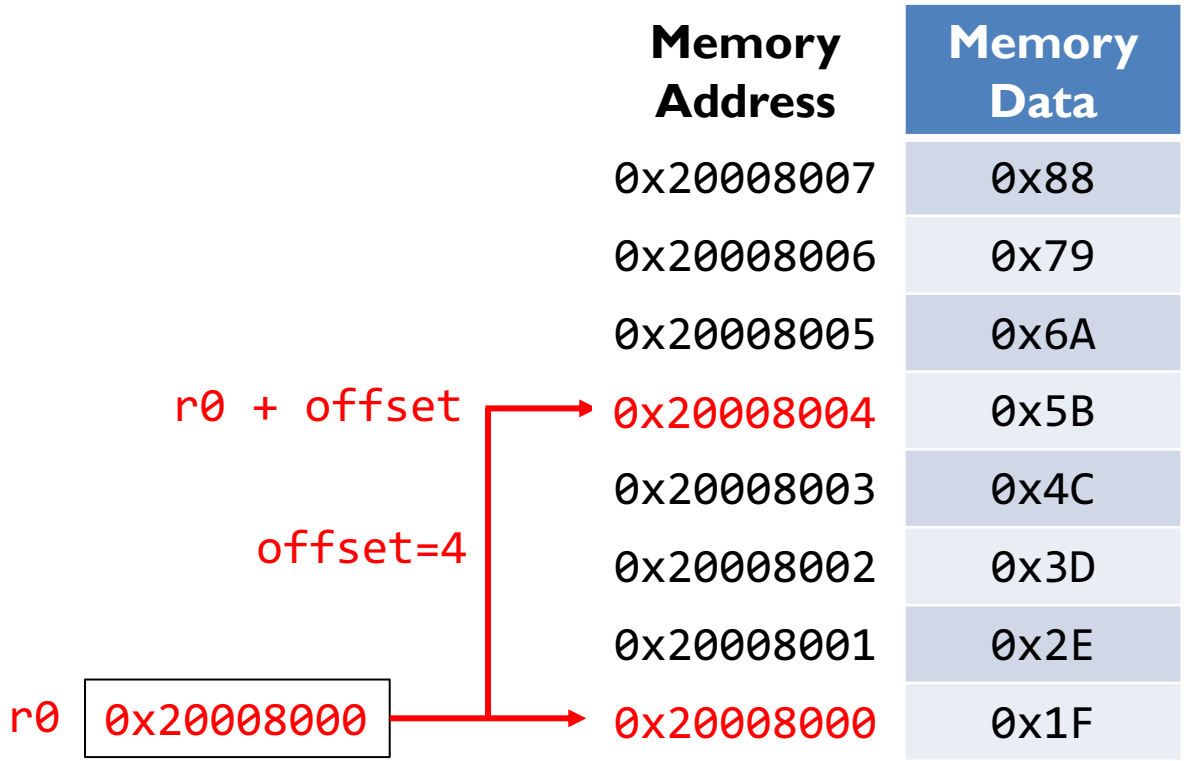
r0 | 0x20008000 → 0x20008000

Pre-index

Pre-Index: LDR r1, [r0, #4]

Offset: range is -255 to +255

Assume: r0 = 0x20008000

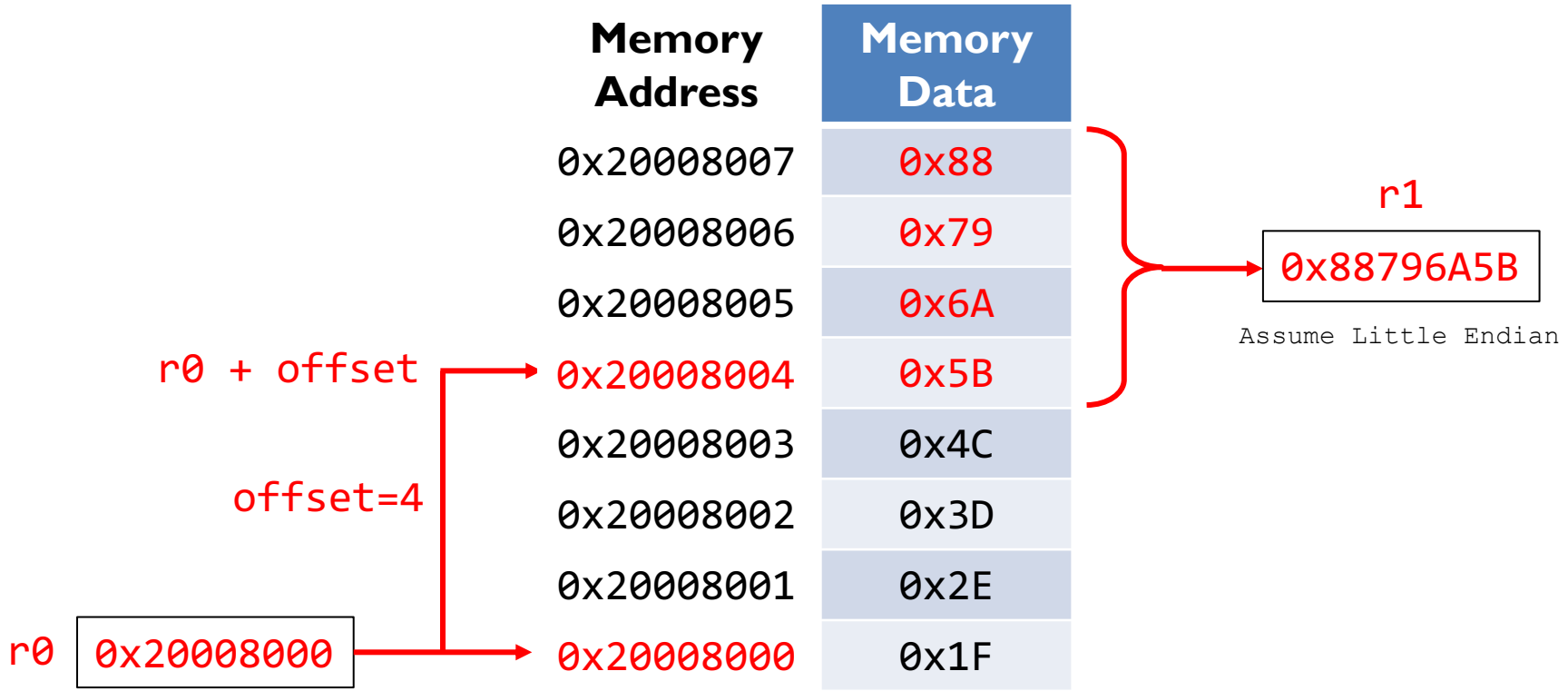


Pre-index

Pre-Index: LDR r1, [r0, #4]

Offset: range is -255 to +255

Assume: r0 = 0x20008000



Accessing an Array

- C code

```
uint32_t array[10];
array[0] += 5;
array[1] += 5;
```

Assume the memory address of the array starts at 0x20008000.

- Pre-index

Assume r0 = 0x20008000.

```
LDR r1, [r0]     ; Read array[0]
ADD r1, r1, #5
STR r1, [r0]     ; Write to array[0]

LDR r1, [r0, #4] ; Read array[1]
ADD r1, r1, #5
STR r1, [r0, #4] ; Write to array[1]
```

Post-index

Post-Index: `LDR r1, [r0], #4`

Offset: range is -255 to +255

Assume: r0 = 0x20008000

Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

Post-index

Post-Index: LDR r1, [r0], #4

Offset: range is -255 to +255

Assume: r0 = 0x20008000

Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

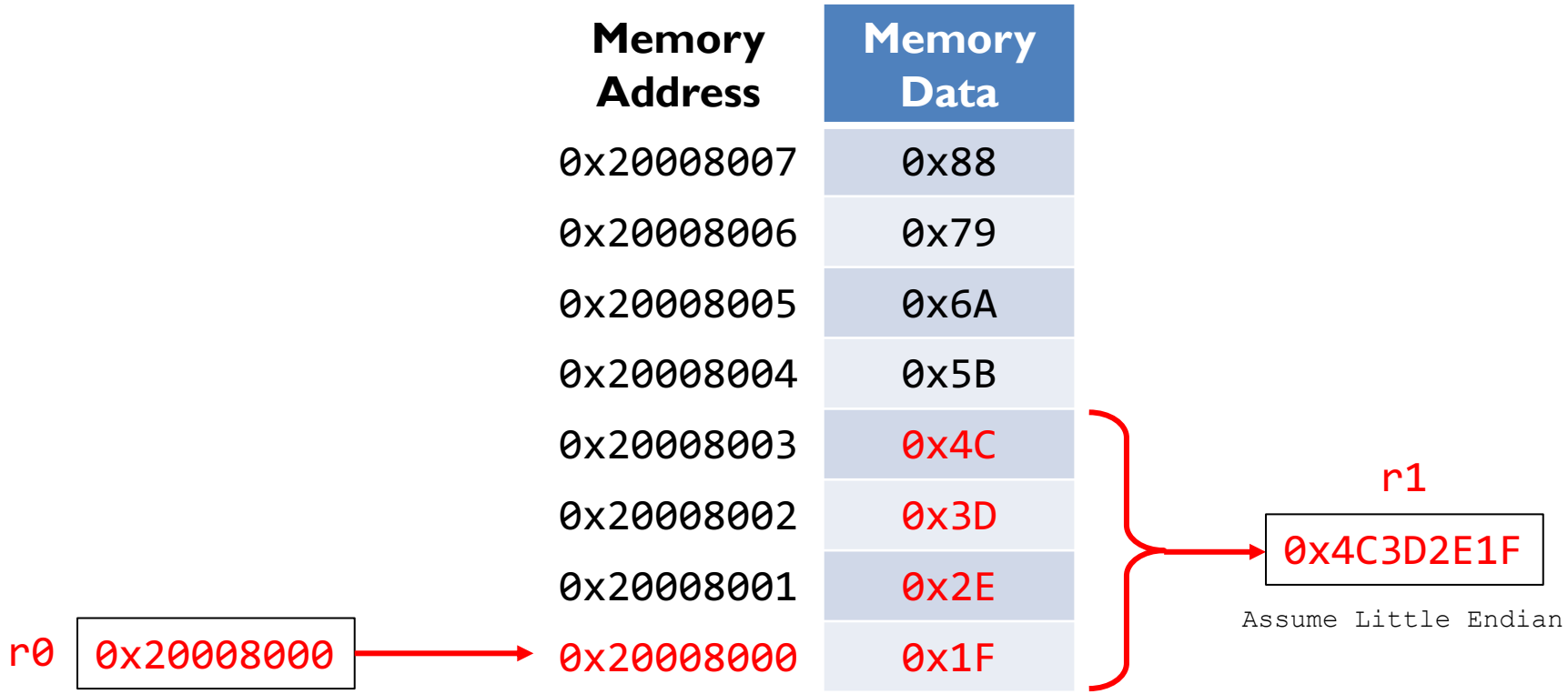
r0 0x20008000 → 0x20008000

Post-index

Post-Index: LDR r1, [r0], #4

Offset: range is -255 to +255

Assume: r0 = 0x20008000



Post-index

Post-Index: LDR r1, [r0], #4

Offset: range is -255 to +255

Assume: r0 = 0x20008000

Update r0 after reading memory

r0 = r0 + offset

Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

r0 | 0x20008004

r1

0x4C3D2E1F

Assume Little Endian

Pre-index with Update

Pre-Index with Update: `LDR r1, [r0, #4]!`

Offset: range is -255 to +255

Assume: r0 = 0x20008000

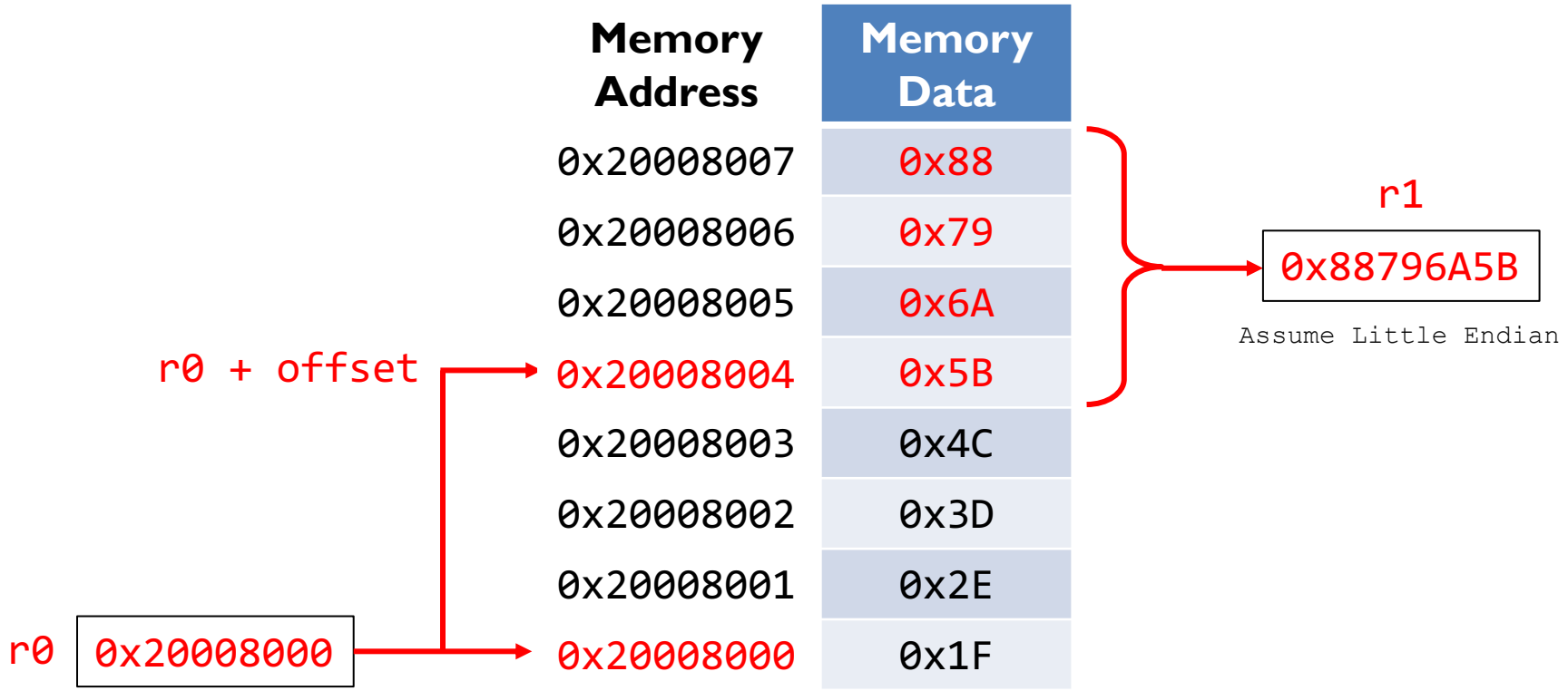
Memory Address	Memory Data
0x20008007	0x88
0x20008006	0x79
0x20008005	0x6A
0x20008004	0x5B
0x20008003	0x4C
0x20008002	0x3D
0x20008001	0x2E
0x20008000	0x1F

Pre-index

Pre-Index with Update: LDR r1, [r0, #4]!

Assume: r0 = 0x20008000

Offset: range is -255 to +255

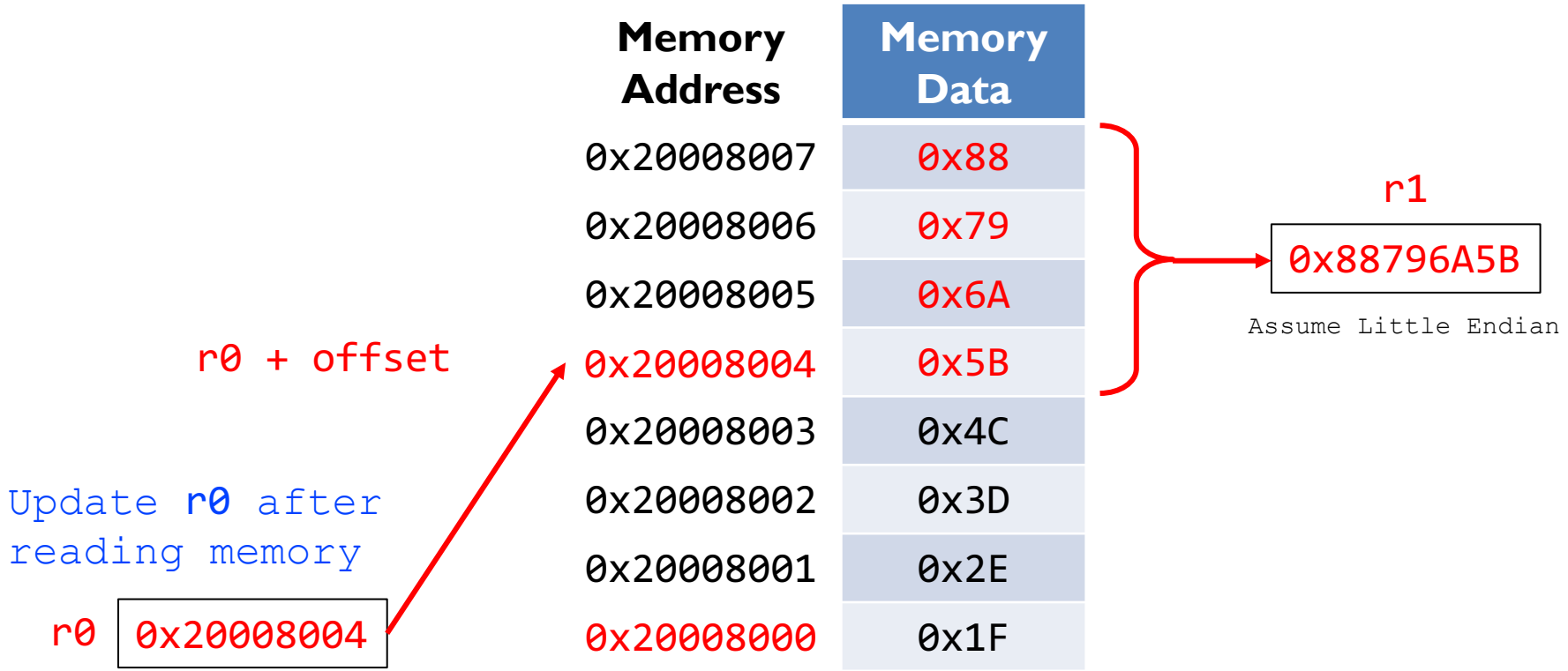


Pre-index

Pre-Index with Update: LDR r1, [r0, #4]!

Assume: r0 = 0x20008000

Offset: **range is -255 to +255**



Summary of Pre-index and Post-index

Index Format	Example	Equivalent
Pre-index	LDR r1, [r0, #4]	r1 ← memory[r0 + 4], r0 is unchanged
Pre-index with update	LDR r1, [r0, #4]!	r1 ← memory[r0 + 4] r0 ← r0 + 4
Post-index	LDR r1, [r0], #4	r1 ← memory[r0] r0 ← r0 + 4

Offset range is -255 to +255

Example

```
LDRH r1, [r0]
; r0 = 0x20008000
```

r1 before load

0x12345678

r1 after load

0x0000CDEF

Memory Address	Memory Data
0x20008003	0x89
0x20008002	0xAB
0x20008001	0xCD
0x20008000	0xEF

Example

```
LDSB r1, [r0]
; r0 = 0x20008000
```

r1 before load

0x12345678

r1 after load

0xFFFFFFEF

Memory Address	Memory Data
0x20008003	0x89
0x20008002	0xAB
0x20008001	0xCD
0x20008000	0xEF

Example

```
STR r1, [r0], #4
; r0 = 0x20008000,  r1=0x76543210
```

r0 before store

0x20008000

r0 after store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

```
STR r1, [r0], #4
; r0 = 0x20008000,  r1=0x76543210
```

r0 before store

0x20008000

r0 after store

0x20008004

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x76
0x20008002	0x54
0x20008001	0x32
0x20008000	0x10

Example

```
STR r1, [r0, #4]
; r0 = 0x20008000,  r1=0x76543210
```

r0 before the store

0x20008000

r0 after the store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

```
STR r1, [r0, #4]
; r0 = 0x20008000,  r1=0x76543210
```

r0 before store

0x20008000

r0 after store

0x20008000

Memory Address	Memory Data
0x20008007	0x76
0x20008006	0x54
0x20008005	0x32
0x20008004	0x10
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

```
STR r1, [r0, #4]!
; r0 = 0x20008000,  r1=0x76543210
```

r0 before store

0x20008000

r0 after store

Memory Address	Memory Data
0x20008007	0x00
0x20008006	0x00
0x20008005	0x00
0x20008004	0x00
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Example

```
STR r1, [r0, #4]!
; r0 = 0x20008000,  r1=0x76543210
```

r0 before store

0x20008000

r0 after store

0x20008004

Memory Address	Memory Data
0x20008007	0x76
0x20008006	0x54
0x20008005	0x32
0x20008004	0x10
0x20008003	0x00
0x20008002	0x00
0x20008001	0x00
0x20008000	0x00

Addressing Modes for Load/Store Multiple Registers

```
STMxx rn{!}, {register_list}
LDMxx rn{!}, {register_list}
```

- xx = IA, IB, DA, or DB

Addressing Modes	Description	Instructions
IA	Increment After	STMIA, LDMIA
IB	Increment Before	STMIB, LDMIB
DA	Decrement After	STMDA, LDMDA
DB	Decrement Before	STMDB, LDMDB

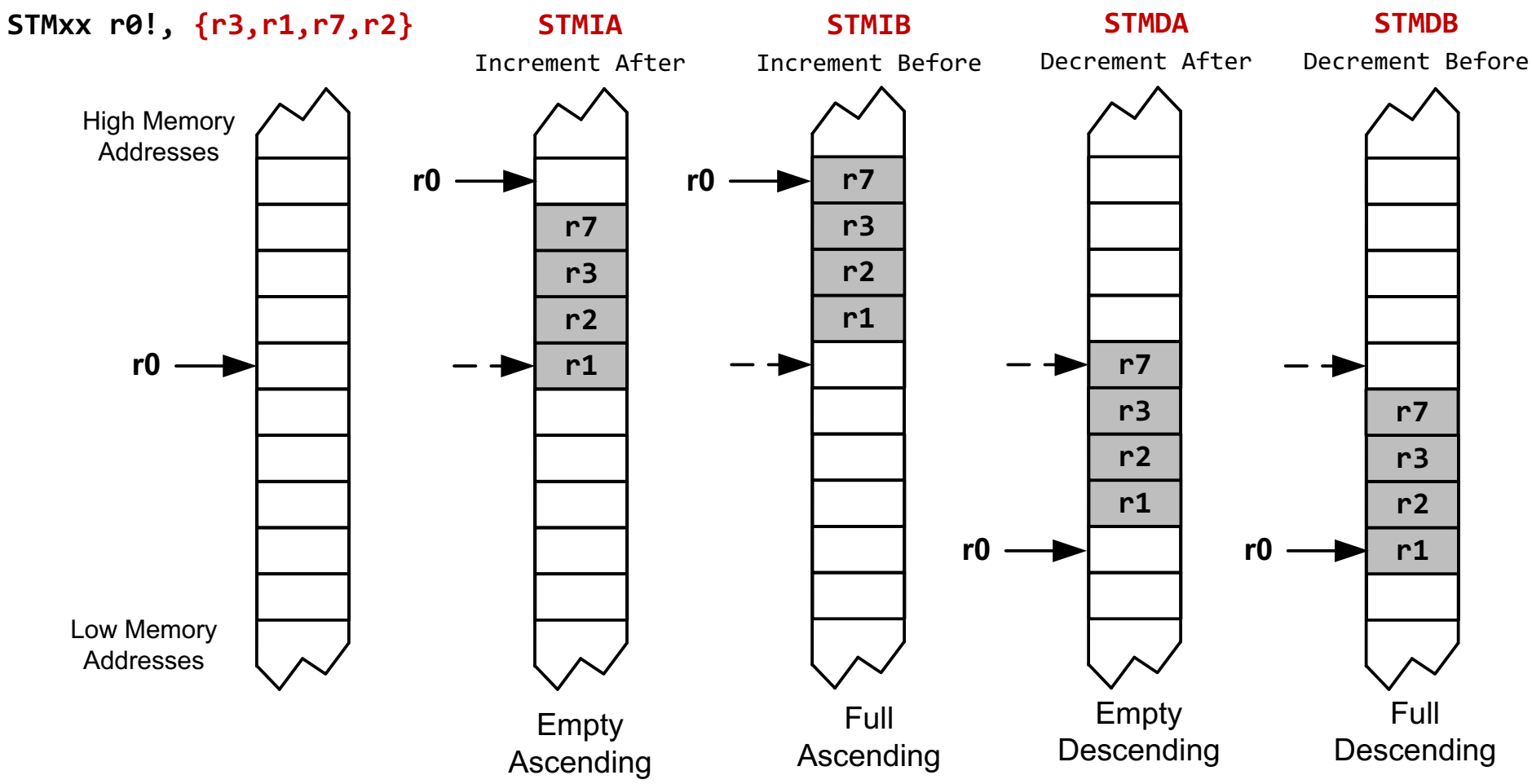
- **IA**: address is incremented by 4 after a word is loaded or stored.
- **IB**: address is incremented by 4 before a word is loaded or stored.
- **DA**: address is decremented by 4 after a word is loaded or stored.
- **DB**: address is decremented by 4 before a word is loaded or stored.

Load/Store Multiple Registers

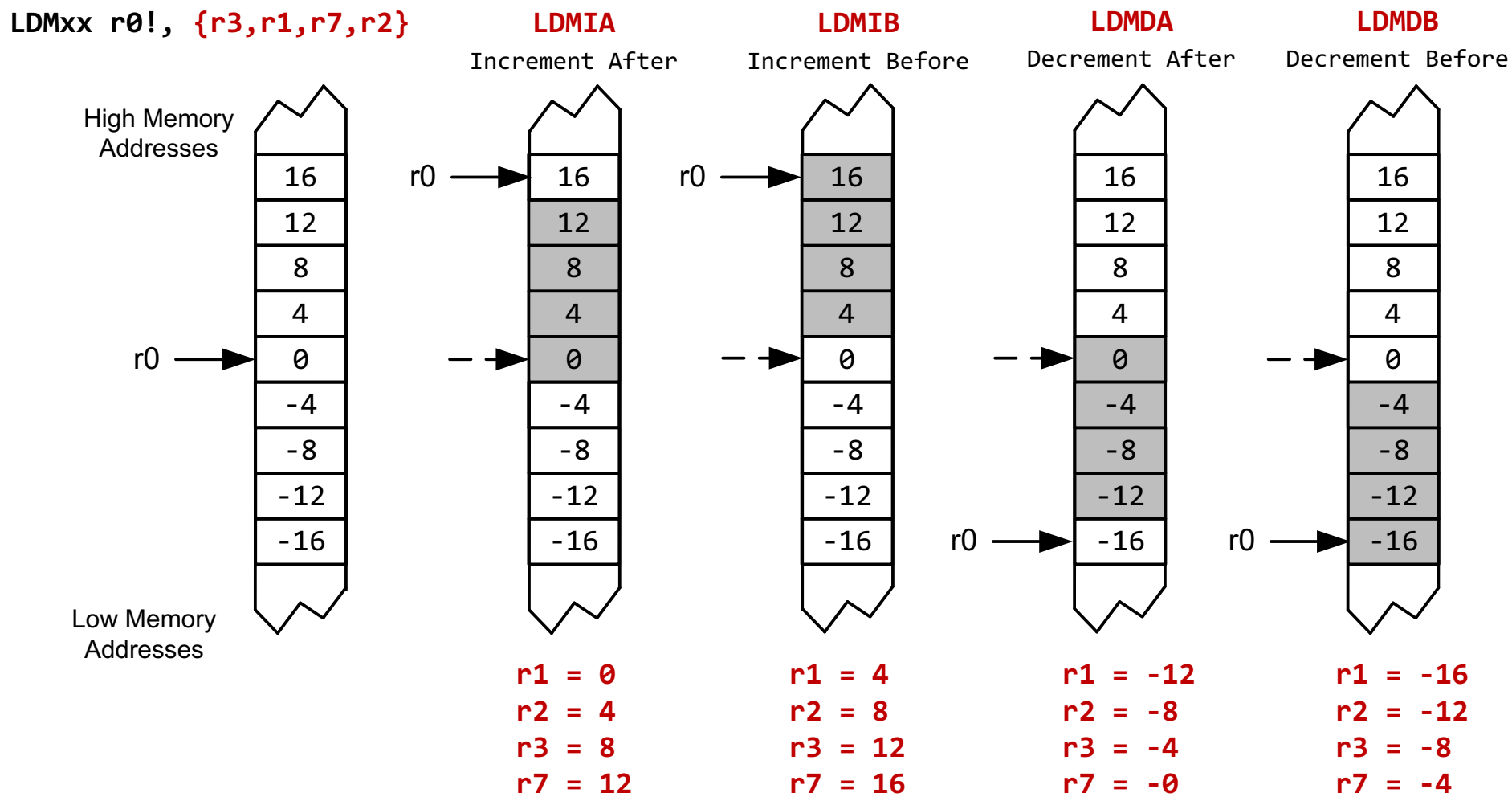
- The following are synonyms.
  - **STM** = **STMIA** (Increment After) = **STMEA** (Empty Ascending)
  - **LDM** = **LDMIA** (Increment After) = **LDMFD** (Full Descending)

- The order in which registers are listed does not matter
  - For STM/LDM, the lowest-numbered register is stored/loaded at the lowest memory address.

Store Multiple Registers



Load Multiple Registers



Pseudo-instructions

- Pseudo instruction: available to use in an assembly program, but not directly supported by hardware.
- Pseudo → not real
- Compilers translate it to one or multiple actual machine instructions
- Pseudo instructions are provided for the convenience of programmers.

LDR Pseudo-instruction

```
LDR Rt, =expr
LDR Rt, =label
```

- If the value of expr can be loaded with **MOV**, **MVN** (16-bit instruction) or **MOVW** (32-bit instruction), the assembler uses that instruction.
- If a valid MOV, MVN, MOVW instruction cannot be used, or if the label_expr syntax is used, the assembler places the constant in a literal pool and generates a PC-relative LDR instruction that reads the constant from the literal pool.

```
LDR r1,=0xFF0   ; loads 0xFF0 into R1
                ; =>  MOV r1,#0xFF0
LDR r2,=0xFFF   ; loads 0xFFF into R2
                ; =>  MOVW r2, #0xFFF
LDR r3,=array   ; loads the address of array into R3
                ; =>  LDR r3,[pc, offset_to_litpool]
                ;     ...
                ;     litpool DCD array
```

Software uses this pseudo instruction to set a register to some value without worrying about the size of the value.

ADR Pseudo-instruction

- **ADR**: loads a program-relative or register-relative address into a register

```
Start   …
        …
        ADR r4,start     ; 32-bit instruction
                         ; => SUB r4,pc,#0xc
```